Zenith: programs, experiments and useful memory

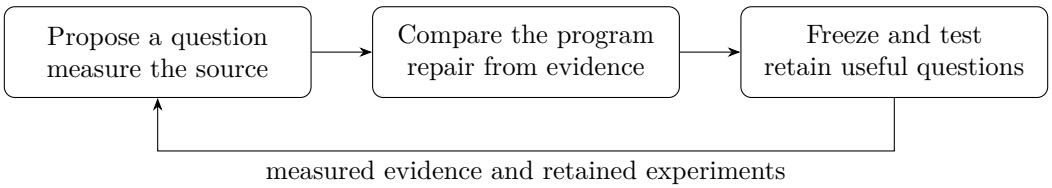
Will Cook • 15 October 2026

Abstract. Zenith asks whether solving one problem can improve how the next is investigated. It retains programs, measured evidence and questions that expose their limits. Three separate mechanisms currently run: model-driven program repair, reuse of generated testers, and an authored continuation learner. Repair alone improves 259/576 later answers to 365/576; added investigation does not beat it. Two generated testers find three of four authored faults; fixed tests find all four. Retained questions help in a controlled memory comparison, while the whole learner still costs more than ordinary alternatives. These results motivate a question about improving future improvement, without demonstrating a new general learning method or recursive self-improvement. We describe the missing connection, a possible mathematics application and the comparisons needed to test them.

1. Programs, evidence and questions. The retained object is (P, E, Q): a program, source-bound observations and executable tests or distinguishing continuations. A useful addition must improve later behavior or reduce the work required to acquire it. Cached answers, learned representations and better investigation procedures require separate controls. Model weights remain fixed in these studies.

Find what needs to change. A wrong answer may reflect discarded history, an incorrect use of sufficient state, or behavior never observed. The archived SQLite program retained pending inserts but ignored them when reading allocation state; another rows-only helper really did discard necessary history. These need different interventions. The records expose the distinction; they do not establish reliable autonomous diagnosis. A model's explanation of a failure remains a hypothesis.

The implemented loop. In the native study, a model receives a task contract, a whole candidate program, examples and the measurements available to its condition. It proposes queries; the host compares candidate and reference answers. A repair request proposes replacement code. Selection preserves every previously correct measured case and the aggregate development result before later panels open. Rejected proposals and failures remain recorded. Passing these finite checks does not prove general correctness. Lab, Oracle and Evolve name execution, review and revision roles; they need not be separate agents.



The client study separately compiles testers from development investigations, then freezes them for later clients whose interface is supplied in advance. The continuation learner below is not called by either model-driven study. A graph can expose dependencies between code and evidence, but it may shrink or disappear when simpler code works better. Its existence is not a result.

2. What the experiments establish. Question memory and learner repair. A continuation asks what a source returns after further actions. Two histories with the same current output can have different future consequences. An authored rational-linear learner uses answers to selected continuations as coordinates, then fits transition and readout maps. Predictive state representations and weighted automata learning are established precedents [1, 2]; Zenith does not discover this model class.

The earlier learner sometimes retained a distinguishing question but discarded the access history witnessing it. Keeping both repairs this defect. The smallest queue example fits in ten source calls and agrees on lengths 0–32. Its five rational-linear coordinates are relative to the imposed class: one nonlinear integer counter represents that queue more simply.

Across eight inspected native configurations, both the repaired refinement and a generic bank of one-symbol questions complete every fit. Generic questions cost 11,379 acquisition calls versus 11,395. Later finite checks retain errors: UTF-8 scores 4,903/4,913; SQLite rowid 190/192 and AUTOINCREMENT 191/192. These are diagnostic fixtures, not eight independent applications.

A separate six-teacher study holds cached answers identical. Retaining acquired questions reduces extension queries by 11.2–63.1% in six declared conditions. Yet every joint lifetime costs 5,427–5,698 source executions, versus 3,071 for separate ordinary modules. Blindly transferring all learned suffixes into the simpler learner costs 3,541 and hurts five of six targets. Exact certificates there concern supplied rational-linear references, not arbitrary native software. Useful questions have not yet made the full architecture economical.

Reusable SQLite testers. Five authored batch clients supply development experience. Two generated testers then run unchanged on five account-transfer clients, four faulty and one correct. Each has six query slots per client; invalid and duplicate queries consume slots. Target execution finishes before a separate 400-case panel opens.

Method	Faults / 4	Native probes	Target model calls
Retained procedure	3	5	0
Ordinary code reuse	3	5	0
Same-history fresh	3	15	19
Cold investigation	3	15	15
Fixed tester	4	7	0

Both generated testers repeat one constant query, ignore feedback and spend ten slots on duplicates. DeepSeek V4.1 Flash on Morph uses 46 calls for $0.007117764, including development and compilation. All three sequential versions, including two earlier unsuccessful attempts, cost $0.019295552 across 133 calls. The family was inspected during engineering. This establishes executable reuse, not adaptive testing or independent transfer. Charges exclude engineering and host costs; these populations are not pooled with the other studies.

Separate investigation diagnostic. On twelve parent episodes from SQLite, LRU and UTF-8 contracts, repair without new investigation improves confirmation from 259/576 to 365/576 answers. Fixed testing scores 364, compiled investigation 354, a direct batch 339, feedback-aware two-turn requests 343 and outcome-masked requests 331. The paired feedback gain is twelve answers on one parent; eleven parents tie. Only five pairs reach a second decision. Eleven parent programs are unique, but there are only three authored mechanisms; case rows are not independent replications. No investigation arm beats repair alone. Ling3.1Flash on DeepInfra uses 130 calls for $0.12982944. Separate qualification, failed host admission and a funding-only continuation retain their own receipts; no failed attempt is erased or counted twice.

3. A direction to test, open to correction. The candidate connection is to retain the experiment that reveals a missing distinction, the histories and outcomes justifying it, the revised program, and a procedure for recognising when a similar investigation would help elsewhere. Selective, economical transfer between representation and investigation remains unbuilt. The current contribution is a repaired learner and reproducible comparisons with informative negative controls.

Relation to earlier work. Learning tests or reusable procedures is already studied in ILF [3], CoverUp [4] and Voyager [5]. STOP [6] applies a code improver to itself; Hyperagents [7] makes the task agent and its improvement machinery editable. Self-editing, a graph or fixed foundation-model weights cannot establish Zenith's novelty. A contribution would require a specified mechanism linking measured distinctions to better subsequent improvement, and evidence beyond these neighbours and strong ordinary alternatives.

Testing improvement of the improver. Let an initial improver produce a successor. Compare both on untouched copies of the same starting solvers and fresh task families under matched lifetime budgets. Then compare reinvesting the successor to produce another generation with continuing the original improver. Keep the evaluator independent of edits; count failed proposals, acquisition, evaluation, execution and human intervention. Remove the learned questions or procedure while retaining the same observations and saved code. Better final answers alone do not show that the improver became better. Any positive result would be bounded by the tested tasks and budgets, without establishing open-ended acceleration.

Why mathematics might help. The Plectis mathematical corpus contains Lean proofs, partial results and open obligations. Its argument-continuation exporter identifies candidate theorems for a required hypothesis and remaining obligations. This suggests decisions to study: prove a prerequisite, try another route, or seek a counterexample. Plectis Atlas helps readers inspect OpenAI's mathematics release and attributed outside assessments; the mathematics and assessments belong to their named authors. Neither project is a demonstrated learning integration with Zenith. A useful record would connect what was known, the action, measured outcome, cost and effect on the target. Finished proofs do not supply missing histories of alternatives. Lean checks a supplied proof; failed search does not establish falsehood, and correct lemmas need evidence of relevance. Begin with cheap software tests; mathematics is a possible harder transfer domain.

Collaboration. I am building Zenith with AI coding agents and learning as I go, with limited experience in this research area. This is my current understanding. I welcome corrections, collaboration and guidance on prior work, evaluations, real tasks and the mathematics connection through the project's issues. Results and replay links retain failures and exact costs; the programme expands the proposed comparisons.

References

[1] M. Littman, R. Sutton and S. Singh. Predictive Representations of State. NeurIPS, 2001.
[2] J. Worrell. Exactly Learning Weighted Automata over a Field. Oxford notes.
[3] J. He et al. Learning to Fuzz from Symbolic Execution. CCS, 2019.
[4] J. A. Pizzorno and E. D. Berger. CoverUp: Effective High Coverage Test Generation for Python. 2025.
[5] G. Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. 2023.
[6] E. Zelikman et al. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation. 2023.
[7] J. Zhang et al. Hyperagents. 2026.